

Quality Concepts

Quality Concepts

- Intro
- Quick Look
- WHAT IS QUALITY?
- SOFTWARE QUALITY
 - Garvin's Quality Dimensions
 - McCall's Quality Factors
 - ISO 9126 Quality Factors
 - [ISO 25010](#)
 - Targeted Quality Factors
 - The Transition to a Quantitative View
- THE SOFTWARE QUALITY DILEMMA
 - "Good Enough" Software
 - The Cost of Quality
 - Risks
 - Negligence and Liability
 - Quality and Security
 - The Impact of Management Actions
- 19.4 – ACHIEVING SOFTWARE QUALITY
 - Software Engineering Methods
 - Project Management Techniques
 - [Machine Learning and Defect Prediction](#)
 - Quality Control
 - Quality Assurance
- SUMMARY

Intro

Learn about the principles, concepts, and techniques that are applied to manage and control software quality.

- Generic characteristics of high-quality **software**
- Conduct effective quality **reviews**
- Software **quality assurance**
- Strategies for **software testing**
- Methods to design **effective test cases**
- Manage and **control changes**
- **Measures and metrics** assess the quality

Intro

By the **1990s**, major corporations **recognized** that **billions of dollars each year** were **being wasted** on software that **didn't deliver** the **features and functionality** that were **promised**.

- Major **software fault** might **cripple important infrastructure**, costing tens of **billions** more
 - **Plagues nearly every organization** that uses computers
 - **Causing lost work hours** during computer downtime
 - **Lost or corrupted data**
 - **Missed sales opportunities**
 - **High IT support-** and **maintenance costs**
 - **Low** customer **satisfaction**

Intro

Experts say...

- Only **3 or 4 defects per 1,000 lines** of code to make a program perform **poorly**.
- Most **programmers inject** about **1 error for every 10 lines** of code
- **Multiply** that **by the millions of lines** of code in many commercial products
- It **costs software vendors** at **least half** their **development budgets** to **fix errors while testing**

Get the picture?

Intro

Who is to blame?

- Customers blame developers
 - Arguing that sloppy practices lead to low-quality software
- Developers blame customers (and other stakeholders)
 - Arguing that **irrational delivery dates** and a **continuing stream of changes** force them to **deliver software before it has been fully validated**

Who's right? Both -and that's the problem 😞

Quick Look

- **What is it?**

- You **know** quality **when** you **see** it
- It can be an **elusive** thing to define
- Quality is something we have **to define**

- **Who does it?**

- Everyone **involved** in the software process is **responsible** for quality

- **Why is it important?**

- Do it right, or do it over again
- Results in **lower costs**, and more importantly, improved time to market

Quick Look

- **What are the steps?**

- 4 activities
 - Proven software engineering **process and practice**
 - Solid project **management**
 - Comprehensive **quality control**
 - Presence of a **quality assurance infrastructure**

- **What is the work product?**

- Software that **meets** its customer's **needs**
- **Provides value to all** who use it

- **How do I ensure that I've done it right?**

- **Examining the results** of all **quality control activities**
- **Measure quality** by **examining errors before** delivery and **defects released** to the field

What Is Quality?

What Is Quality?

Quality...

- You know what it is, yet you don't know what it is...
- If you can't say what Quality is...
 - How do you know what it is?
 - How do you know that it even exists?

...then for all practical purposes it doesn't exist at all.

But for all practical purposes it really does exist!

What Is Quality?

“Quality is a complex and multifaceted concept”
which can be described from 5 different points of view.
- by David Garvin of the Harvard Business School

- Transcendental view
- User view
- Manufacturer's view
- Product view
- Value-based view

What Is Quality?

Transcendental view

- Something you immediately **recognize**, but **cannot** explicitly **define**

User view

- If a product **meets** an end user's **specific goals**

Manufacturer's view

- If the product **conforms** original **specification**

Product view

- Can be **tied to inherent characteristics** (e.g. **functions and features**) of a product

Value-based view

- Measures on **how much a customer is willing to pay** for a product

What Is Quality?

In reality, quality encompasses all of these views and more...

“Quality is important, but if the user isn't satisfied, nothing else really matters.”

- by Robert Glass

Argues that a more **"intuitive"** relationship is in order:

user satisfaction =

compliant product + good quality + delivery within budget and schedule

"A product's quality is a function of how much it changes the world for the better."

- by DeMarco

Software Quality

Software Quality

How do we define software quality?

In the most general sense:

“An effective software process applied in a manner that creates a useful product that provides measurable value for those who produce it and those who use it”

Could be modified or extended and **debated endlessly...**

Software Quality

For the purposes of this book, the definition serves to emphasize 3 important points:

1. An ***effective software process*** establishes the infrastructure that supports any effort at building a high-quality software product
2. A ***useful product*** delivers the content, functions, and features that the end user desires, but as important, it delivers these assets in a reliable, error-free way
3. By ***adding value for both the producer and user*** of a software product, high-quality software provides benefits for the software organization and the end-user community

Garvin's Quality Dimensions

Although **Garvin's 8 dimensions of quality** were not developed specifically for software, they can be applied when software quality is considered.

- **Garvin's quality dimensions provide** you with a **"soft" look** at software quality

You also need a set of "hard" quality factors:

1. Factors that can be **directly** measured
2. Factors that can be measured only **indirectly**

Garvin's Quality Dimensions

Garvin's 8 dimensions of quality:

1. Performance quality
2. Feature quality
3. Reliability
4. Conformance
5. Durability
6. Serviceability
7. Aesthetics
8. Perception

McCall's Quality Factors

McCall, Richards, and Walters propose a useful categorization of factors that affect software quality.

Focus on 3 important aspects of a software product:

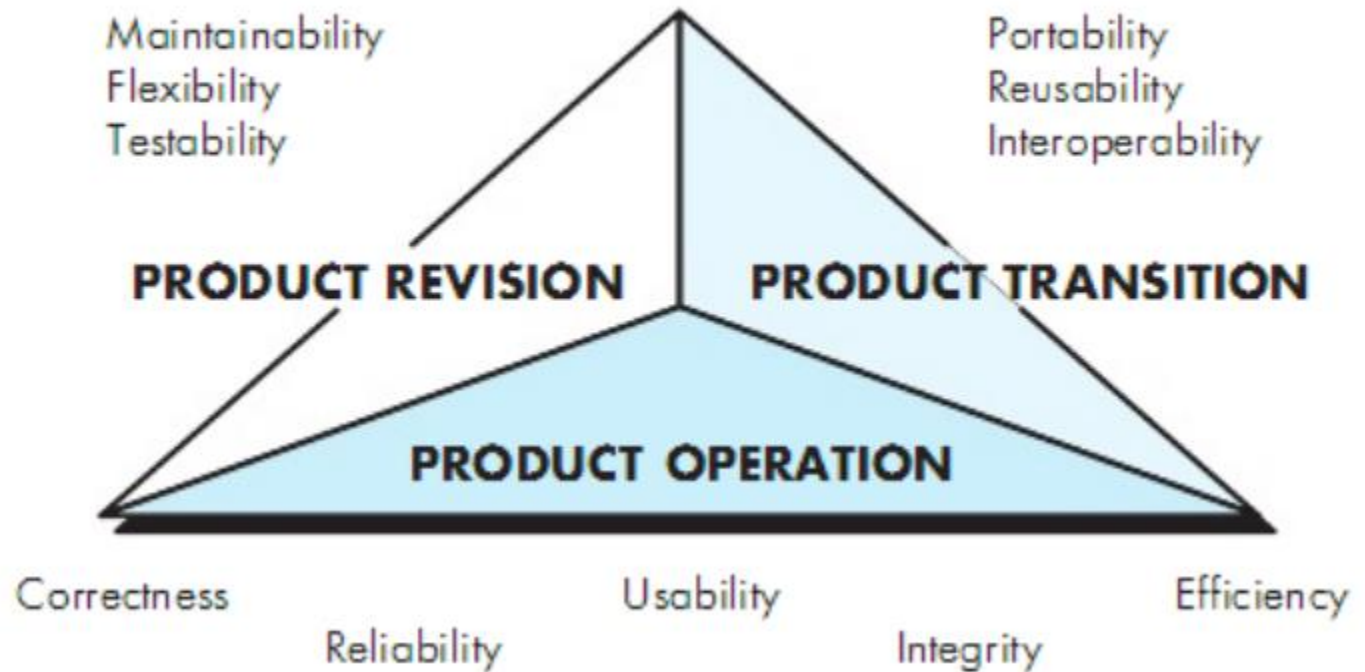
1. Its **operational** characteristics
2. Its **ability to** undergo **change**
3. Its **adaptability** to new environments

Many of the metrics **can be measured only indirectly.**

McCall's Quality Factors

FIGURE 19.1

McCall's
software
quality factors



ISO 9126 Quality Factors

The standard identifies **6 key quality attributes** developed in an attempt to identify the key quality attributes for computer software:

1. Functionality
2. Reliability
3. Usability
4. Efficiency
5. Maintainability
6. Portability

Provides a worthwhile **basis for indirect measures** and an **excellent checklist** for assessing the quality of a system.

ISO 25010

- Quality in Use Model
 - Effectiveness
 - Efficiency
 - Satisfaction
 - Freedom from Risk
 - Context Coverage
- Product Quality
 - Functional Suitability
 - Performance Efficiency
 - Compability
 - Usability
 - Reliability
 - Security
 - Maintainability
 - Portability

<https://www.iso.org/standard/35733.html>

Targeted Quality Factors

- The **quality dimensions and factors** presented in Sections 19.2.1 and 19.2.2 focus on the software as a whole and **can be used as a generic indication of the quality** of an application.
- A software **team can develop a set of quality characteristics and associated questions** that would probe the degree to which each factor has been satisfied.

See p.419 for example – “Interface”

Intuitiveness – Efficiency – Robustness – Richness

To conduct your assessment, you'll need to address specific, measurable (or at least, recognizable) attributes of the interface.

Intuitiveness—Efficiency—
Robustness—Richness

Intuitiveness–Efficiency–Robustness–Richness

Intuitiveness. The degree to which the interface follows expected usage patterns so that even a novice can use it without significant training.

- Is the interface layout **conducive to easy understanding**?
- Are interface **operations easy to locate and initiate**?
- Does the interface **use a recognizable metaphor**?
- Is input specified to **economize keystrokes or mouse clicks**?
- Does the interface **follow the three golden rules**? [Link](#)
- Does **aesthetics aid in understanding and usage**?

Intuitiveness–**Efficiency**–Robustness–Richness

Efficiency. The degree to which operations and information can be located or initiated.

- Does the interface **layout and style allow a user to locate operations and information** efficiently?
- Can a **sequence of operations** (or data input) be performed **with an economy of motion**?
- Are output data or content **presented so that it is understood immediately**?
- Have hierarchical operations been organized in a way that **minimizes the depth to which a user must navigate** to get something done?

Intuitiveness–Efficiency–Robustness–Richness

Robustness. The degree to which the software handles bad input data or inappropriate user interaction.

- Will the software **recognize the error** if data values are at or just **outside prescribed input boundaries**?
More importantly, will the software **continue to operate without failure or degradation**?
- Will the interface **recognize common cognitive or manipulative mistakes** and explicitly **guide the user back on the right track**?
- Does the interface provide **useful diagnosis and guidance when an error condition** (associated with software functionality) is uncovered?

Intuitiveness–Efficiency–Robustness–**Richness**

Richness. The degree to which the interface provides a rich feature set.

- Can the interface be **customized to the specific needs** of a user?
- Does the interface **provide a macro capability that enables a user to identify a sequence of common operations with a single action or command?**

The Software Quality Dilemma

The Software Quality Dilemma

Bertrand Meyer discusses what is called the **quality dilemma**...

...people in industry try to get to that **magical middle ground** where the product is **good enough not to be rejected** right away, such as during evaluation, but **also not** the object of so **much perfectionism and so much work** that it would **take too long or cost too much** to complete.

"Good Enough" Software

Accept the argument made by **Meyer**...

- Is it **acceptable** to produce "**good enough**" software?

The **answer** to this question **must be yes**.

- Major software **companies does it every day**
- Some of the functions and features delivered in **Version 1.0** may **not be of the highest quality** and **plan** for **improvements** in **Version 2.0**

"Good Enough" Software

What is "good enough"?

It is true that "good enough" may work in some application domains and for a few major software companies.

- A problem for small companies?
- E.g., Real-time embedded software or build application software that is integrated with hardware (e.g., automotive software, telecommunications software), delivering software with known bugs can be negligent and open your company to **expensive litigation**

In some cases, it can even be criminal 😞

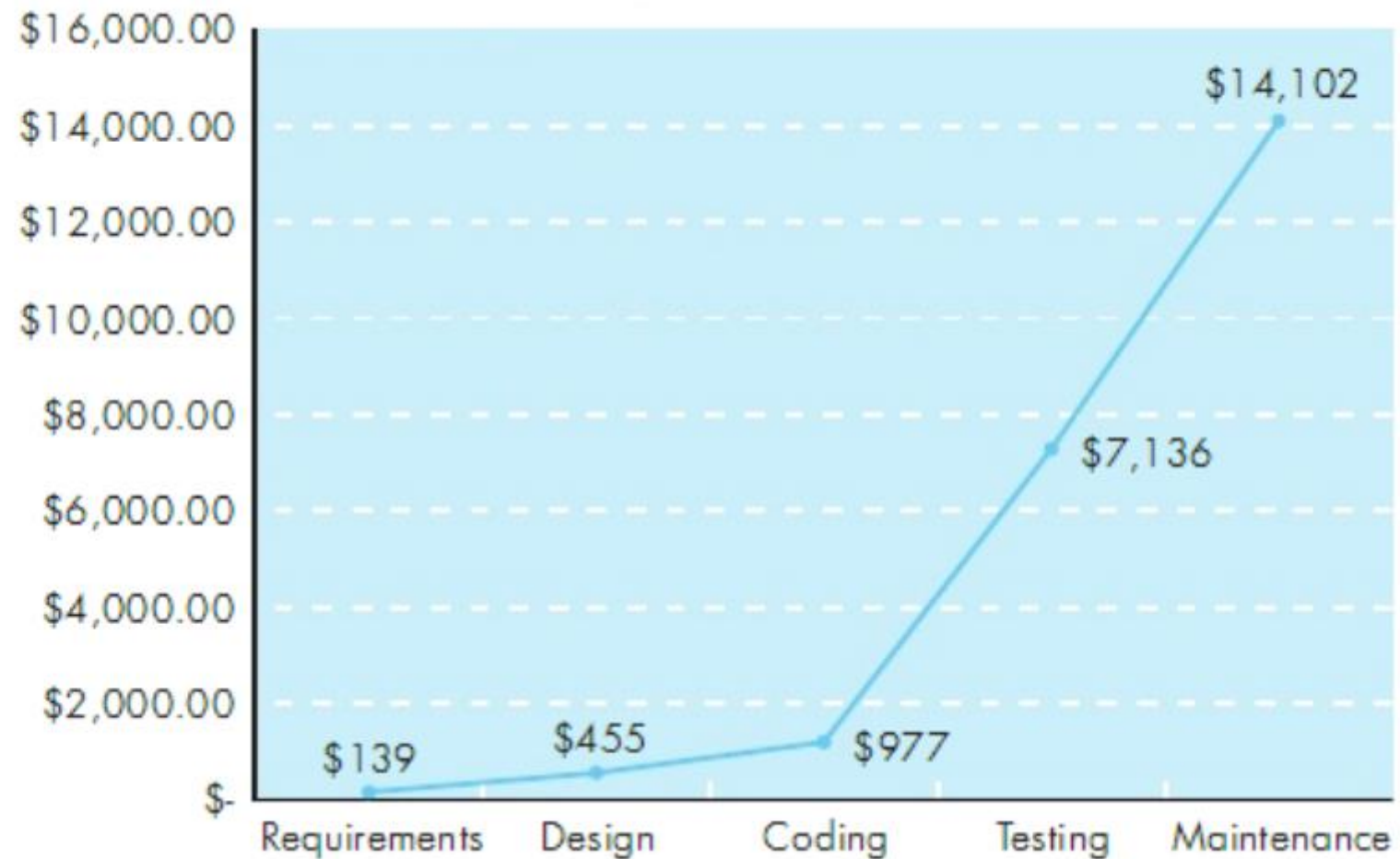
The Cost of Quality

The Cost of Quality

FIGURE 19.2

Relative cost
of correcting
errors and
defects

Source: Adapted
from [Boe01b].



The Cost of Quality

Quality has a cost, but lack of quality also has a cost.

- Which cost should we be worried about?

Understand both...

- **“The cost of quality”** can be divided into costs associated with
 - Prevention
 - Appraisal
 - Failure - Internal & External
- The **cost of low-quality** software

The Cost of Quality

- Prevention
 - Management activities
 - Technical activities
 - Test planning
 - All training associated with these activities
- Appraisal
 - Technical reviews
 - Data collection and metrics evaluation
 - Testing and debugging

The Cost of Quality

- Failure – Internal (error)
 - Perform rework (repair) to correct
 - When rework inadvertently generates side effects that must be mitigated
 - Collection of quality metrics that allow an organization to assess the modes of failure
- Failure – External (defect), examples of costs:
 - Complaint resolution
 - Product return and replacement
 - Help line support
 - Labour costs associated with warranty work
 - A poor reputation and the resulting loss of business

Bad things happen when low-quality software is produced 😞

The Cost of Quality

The relative **costs to find and repair** an error or defect **increase dramatically as we go from prevention to detection** to internal failure to external failure costs.

According to industry average data:

- **Correct a defect during code generation** is approximately **\$977 per error**
- **Correct the same error** if it is discovered **during system testing** is **\$7,136 per error**

Considers a **large application that has 200 errors** introduced during coding.

Risks

Downside of poorly designed and implemented applications does not always stop with dollars and time...

- Throughout the month of November 2000 at a hospital in Panama, 28 patients received massive overdoses of gamma rays during treatment for a variety of cancers.
- In the months that followed, 5 of these patients died from radiation poisoning and 15 others developed serious complications.

What caused this tragedy?

- A software package, developed by a U.S. company, was modified by hospital technicians to compute modified doses of radiation for each patient.
- 3 Panamanian medical physicists, who tweaked the software to provide additional capability, were charged with second-degree murder.
- The US company was faced with serious litigation in two countries.

Poor quality leads to risks, some of them very serious.

Negligence and Liability

The story is all too common...the system might support:

- Corporate function, e.g. pension management
- Governmental function, e.g. healthcare administration or homeland security

Work begins with the best of intentions on both sides...

- By the time the system is delivered...
 - Things have **gone bad**
 - The system **is late**
 - **Fails to deliver** desired features and functions
 - Is **error-prone**
 - Does **not meet** with customer **approval**

Litigation ensures...the “quality of the delivered system” comes into question.

Quality and Security

Stated simply, **software that does not exhibit high quality is easier to hack**, and as a consequence, **low-quality software** can indirectly **increase the security risk** with all of its attendant costs and problems.

- To build a secure system, you must **focus on quality**, and that focus must begin during design
- The **earlier** you find the software problem, the **better**

And there are 2 kinds of software problems:

1. **Bugs** - Implementation problems
2. **Software flaws** - Architectural problems in the design

People pay too much attention to bugs and not enough on flaws.

The Impact of Management Actions

Software quality is often **influenced** as much **by management decisions** as it is by technology decisions.

- **Estimation** decisions
 - If a delivery date is irrational, it is important to hold your ground. Explain **why you need more time**, or alternatively, **suggest a subset of functionality** that can be delivered (with high quality) **in the time allotted**
- **Scheduling** decisions
 - Quality suffers as a consequence, “A” may have defects that are hidden, **only to be discovered much later**
- **Risk-oriented** decisions
 - Our advice: taking the time to **do it right is almost never the wrong decision**
 - **Have a way of handling things** that do go wrong

Achieving Software Quality

Achieving Software Quality

Software quality doesn't just appear.

- It is the result of:
 - Good project management
 - Solid software engineering practice

Applied within the context of 4 broad activities:

1. Software Engineering Methods
2. Project Management Techniques
3. Quality Control
4. Quality Assurance

Software Engineering Methods

- You must understand the problem to be solved

Use **concepts and methods** that can lead to a **reasonably complete understanding of the problem** and a **comprehensive design** that **establishes a solid foundation** for the construction activity – Part 2

*“If we can really understand the problem, the answer will come out of it, because **the answer is not separate from the problem**”*

- by Jiddu Krishnamurti

Project Management Techniques

1. Use **estimation to verify** that **delivery dates are achievable**
2. **Schedule dependencies are understood** and the **team resists** the temptation to use **shortcuts**
3. **Risk planning is conducted**

In addition, the project plan should include explicit techniques for quality and change management – Part 4

Machine Learning and Defect Prediction

Defect Prediction

- **Using a statistical technique to examine the relationship among combinations of software metrics** and software components containing known defects
- **Can reduce costs and development times**

Machine Learning / AI

- Ability to **learn and improve from experience** without being explicitly programmed
- **Accesses data and use it to learn** from themselves
- Can **automate** the process of **discovering predictive relationships** between software metrics and defective components

A big part of what modern scientists do

Quality Control

Use **actions** that **help to ensure** that each work product meets its **quality goals**:

- **Models** are **reviewed**
- **Code** should be **inspected**
- **Testing** steps are **applied**

A combination of measurement and feedback that allows tuning the process.

Quality Assurance

Establish the infrastructure that supports:

- Solid software engineering methods
- Rational project management
- Quality control actions
- In addition, quality assurance consists of a set of auditing and reporting functions.

The goal of quality assurance: To provide management and technical staff with the data necessary to be informed about product quality.

If problems are identified: It is management's responsibility to address the problems and apply the necessary resources.

Summary

Every software organization is faced with the software quality dilemma.

Regardless of the approach that is chosen, quality does have a cost that can be discussed in terms of:

- **Prevention**

- All software engineering **actions that are designed to prevent defects** in the first place

- **Appraisal**

- Are associated with those **actions that assess software work products** to determine their quality

- **Failure**

- Encompass the **internal price of failure** and the external effects of poor quality

Summary

Software quality is achieved through the application of:

- **Software engineering methods**
- **Solid management practices**
- **Comprehensive quality control**

All supported by a software quality assurance infrastructure.

Questions?

Feedback?